

1 Literature Review

1.1 Introduction and Scope

One of the central challenges in modern machine learning research is understanding how large language models represent and process information internally. Although these models demonstrate increasingly strong capabilities, their internal computations remain difficult to interpret because individual neurons often respond to multiple seemingly unrelated inputs. This makes neuron-level inspection an unreliable way to recover the concepts a model has learned. Recent work in mechanistic interpretability has begun to address this opacity by developing methods that operate at the level of features rather than neurons, with sparse autoencoders emerging as one of the most promising tools for this purpose [6, 22].

At the same time, the models most users interact with are not raw pre-trained systems. Before deployment, they typically undergo post-training procedures such as supervised fine-tuning (SFT) and reinforcement learning from human feedback (RLHF), which reshape their behaviour to make them more helpful, aligned, and conversational [18]. What remains less clear is how deeply these behavioural improvements alter the model’s internal representations. This literature review therefore examines three connected strands of research: interpretability work on superposition and sparse autoencoders, alignment work on post-training, and emerging mechanistic studies of what post-training changes internally. A concise summary of the most directly relevant studies is provided in Appendix A.

1.2 Superposition and Sparse Autoencoders

Neural networks face a basic representational constraint: they must encode a very large number of concepts using a comparatively limited number of dimensions. Elhage et al. [6] show that one way networks resolve this tension is through *superposition*, in which many features are stored as nearly orthogonal directions in activation space rather than being assigned cleanly to individual neurons. This helps explain why neurons are frequently polysemantic: a single neuron can participate in representing multiple unrelated concepts, making direct neuron-level

interpretation both noisy and incomplete.

Sparse autoencoders (SAEs) were introduced as a practical way to address this problem by decomposing dense activations into a sparse set of latent features that are often more interpretable than the original neurons. Rather than asking what one neuron means, SAEs attempt to recover a higher-dimensional feature dictionary in which individual features correspond more closely to coherent concepts. This approach has substantially advanced feature-level interpretability in large language models. In particular, Templeton et al. [22] applied SAEs at scale to Claude 3 Sonnet and reported millions of interpretable features. Importantly, they also showed that manipulating these features could causally change model behaviour, which suggests that the recovered feature dictionaries are not merely descriptive artefacts but capture functionally meaningful structure inside the model.

Taken together, this line of work provides both the theoretical and methodological foundation for the present study. If superposition obscures the model’s conceptual organisation at the neuron level, and if SAEs can recover a more interpretable feature basis, then SAEs offer a viable way to compare how a model’s internal feature landscape differs before and after post-training.

1.3 Post-Training and the Superficial Alignment Hypothesis

Modern language models are typically transformed into usable assistants through post-training. Two of the most influential post-training stages are supervised fine-tuning, which teaches the model to follow curated instruction-response examples, and RLHF, which further shapes responses toward human preferences [18]. This pipeline has been shown to produce large gains in helpfulness, harmlessness, and instruction-following behaviour even without changing the base architecture.

What is less clear is whether post-training creates genuinely new capabilities or mainly changes how pre-existing capabilities are expressed. Zhou et al. [23] argue for a strong version of the latter view. Their *LIMA* results suggest that fine-tuning on only 1,000 carefully curated examples, without reinforcement learning, can already yield strong alignment-style improvements. Their interpretation is that most knowledge and core capability are established during

pre-training, while post-training primarily adjusts style, formatting, and behavioural presentation. This idea is closely related to what is often called the *superficial alignment hypothesis*: the claim that post-training modifies the surface behaviour of a model more than its underlying competence.

If this interpretation is correct, then alignment should not be understood primarily as writing new knowledge into the model. Instead, it would act more like a behavioural interface layered onto existing representations. That raises a mechanistic question that remains insufficiently answered in the literature: what, precisely, changes at the representational level when post-training improves instruction-following and refusal behaviour?

1.4 Mechanistic Effects of Post-Training

Recent mechanistic studies suggest that post-training may leave much of the model’s internal structure intact while selectively modifying a smaller set of behaviourally important mechanisms. Du et al. [5] examine post-training through four lenses: knowledge representation, truthfulness, refusal behaviour, and confidence calibration. Their results indicate that the effects of post-training are uneven rather than uniform. Factual knowledge storage appears largely preserved, and truthfulness-related representations remain substantially similar across base and post-trained models. By contrast, refusal behaviour changes more noticeably, with post-training strengthening activation-space directions associated with rejecting harmful or disallowed requests.

These findings support the view that post-training often works by adjusting control-related mechanisms rather than rebuilding the model from first principles. In other words, pre-training may establish a large share of the representational substrate, while post-training amplifies, suppresses, or reroutes particular behavioural tendencies. However, the literature has not yet fully connected this insight to feature-level interpretability. Existing mechanistic analyses identify broad representational continuities and some behaviour-specific differences, but they do not yet provide a systematic account of how the interpretable feature dictionary itself changes across training stages.

1.5 Research Gap

The literature establishes two important points. First, SAEs now make it possible to recover large numbers of interpretable features from language models and to test some of those features causally [22]. Second, post-training can substantially alter outward behaviour while leaving much of the internal representational structure learned in pre-training relatively intact [5, 23]. The gap lies in combining these two developments.

What remains underexplored is how the SAE feature landscape changes across post-training. Does post-training create genuinely new features for behaviours such as instruction-following and refusal? Does it suppress pre-existing harmful or unaligned features? Or does it leave the underlying feature set mostly intact while changing when and how those features are activated? Answering these questions would provide a more mechanistic account of the relationship between pre-training, alignment, and behavioural change.

This study addresses that gap by comparing the feature dictionaries of base and instruction-tuned variants of the same model family using pre-trained sparse autoencoders. The objective is to determine whether post-training primarily creates new features, suppresses existing ones, or reorganises the activation patterns of an already established feature landscape.

2 Methodology

This chapter describes the experimental design, data sources, and analytical methods used to investigate how post-training techniques reshape the feature landscape of large language models. The methodology proceeds in five stages: first, activations are extracted from both the base and instruction-tuned variants of Gemma 2 2B on identical input sequences; second, pre-trained Sparse Autoencoders from the Gemma Scope suite decompose these activations into interpretable features; third, features are matched across models using decoder similarity and activation correlation; fourth, matched and unmatched features are labelled and categorised through automated interpretability; and fifth, selected features are causally verified through activation steering. Each stage is detailed below, with formal definitions, justifications, and explicit assumptions.

2.1 Experimental Overview and Research Design

The central research question asks how the feature landscape differs between the base version of a model and its post-trained snapshot. To answer this, the methodology compares interpretable features extracted from two versions of the same model architecture—one that has only undergone pre-training and one that has additionally been fine-tuned and aligned through instruction tuning and reinforcement learning from human feedback.

The experimental design follows a comparative, observational structure. Rather than training models from scratch or performing controlled fine-tuning, this study leverages an existing pair of publicly available model checkpoints that differ only in whether post-training was applied. By holding architecture and pre-training constant, any systematic differences in feature representations can be attributed to the post-training process.

The analysis operates at three levels of granularity. The macroscopic level examines how the overall feature landscape changes across all layers, quantifying what proportion of features are preserved, newly created, or lost. The mesoscopic level categorises features thematically—into groupings such as syntactic, semantic, safety-relevant, and formatting features—and examines how each category is affected by post-training. The microscopic level investigates individual features of interest through detailed case studies, verifying their causal role in model behaviour through activation steering experiments.

The experimental pipeline is illustrated across three figures. Figure 1 shows the data and activation collection stage, where identical input sequences are processed by both model variants and decomposed into sparse features via their respective Gemma Scope SAEs. Figure 2 shows the feature comparison stage, where sparse feature vectors are matched across models and classified into four categories before undergoing automated labelling. Figure 3 shows the validation and results stage, where categorised features feed into quantitative metrics and causal verification experiments, converging into an assessment of the wrapper hypothesis.

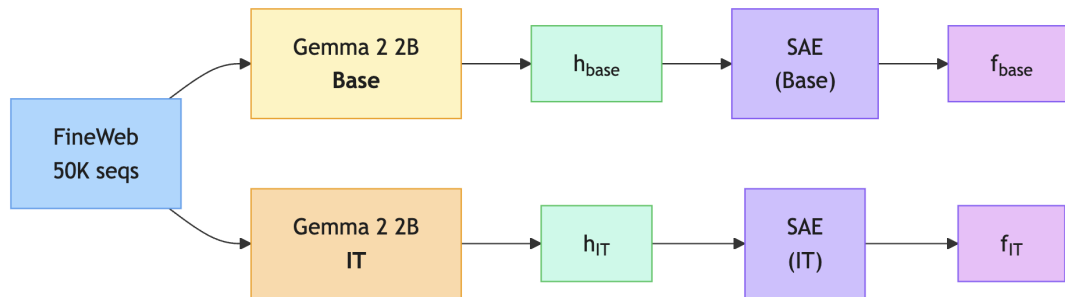


Figure 1: Data and activation collection pipeline. Identical input sequences from FineWeb are processed by both the base and instruction-tuned Gemma 2 2B models, producing residual stream activations that are decomposed into sparse feature vectors by their respective Gemma Scope SAEs.

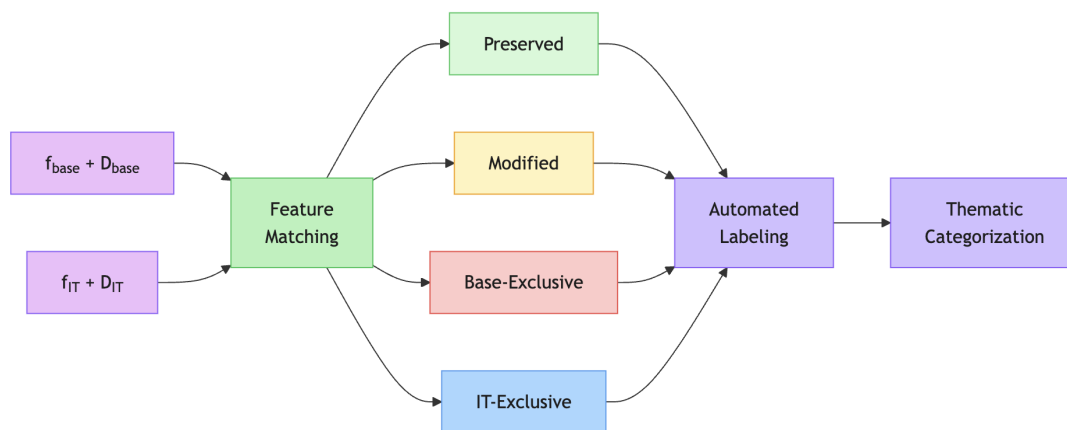


Figure 2: Feature comparison pipeline. Sparse feature vectors and decoder matrices from both models are fed into the feature matching procedure, which classifies features into four categories (preserved, modified, base-exclusive, IT-exclusive) before automated labelling and thematic categorisation.

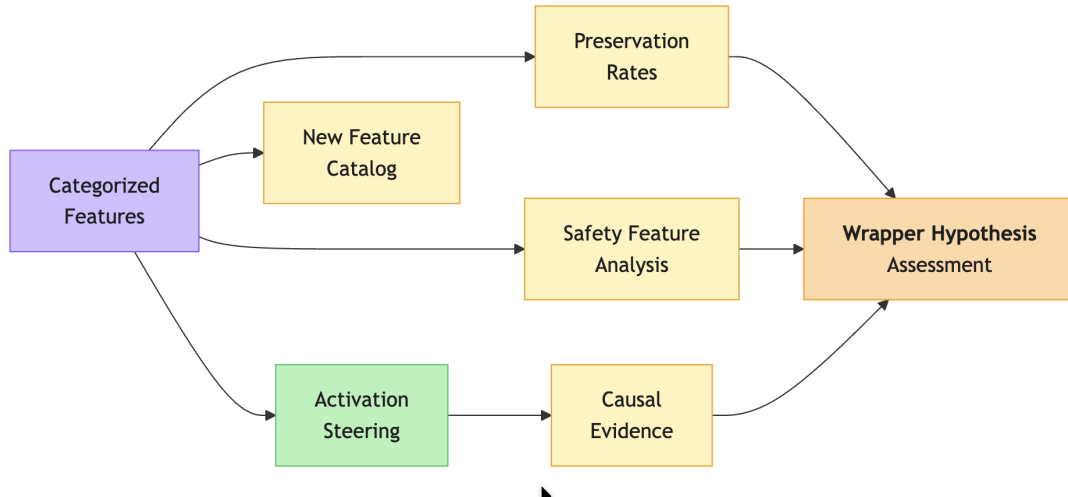


Figure 3: Validation and results pipeline. Categorised features feed into quantitative metrics (preservation rates, new feature catalogue, safety feature analysis) and causal verification via activation steering, converging into an overall assessment of the wrapper hypothesis.

2.2 Data Sources and Description

This study relies on three primary data sources: the language models under comparison, the Sparse Autoencoders used to decompose their activations, and the text corpora used as inputs during analysis.

2.2.1 Language Models: Gemma 2 2B

The models under investigation are the base (pre-trained) and instruction-tuned variants of Gemma 2 2B, developed and released by Google DeepMind [8]. Gemma 2 2B is a dense transformer with 2.6 billion parameters, a hidden dimension of 2304, and 26 layers. The architecture follows a standard transformer decoder with grouped-query attention, rotary positional embeddings, and a vocabulary of 256,000 tokens. Both model variants are publicly available on HuggingFace under open licenses.

The base model (Gemma-2-2B) was trained on approximately 2 trillion tokens of primarily English web data, code, and mathematical content. It represents the output of the pre-training stage alone and produces text through next-token prediction without any explicit behavioural alignment.

The instruction-tuned model (Gemma-2-2B-IT) shares the same architecture and pre-training

foundation but has additionally undergone supervised fine-tuning on curated instruction-response pairs followed by reinforcement learning from human feedback (RLHF). This process is designed to make the model follow instructions, produce helpful and harmless responses, and adopt a conversational format. The IT model’s post-training details follow the procedures described in the Gemma 2 technical report [8].

The choice of Gemma 2 2B is motivated by three considerations. First, the availability of both base and instruction-tuned variants with identical architecture enables controlled comparison. Second, Google DeepMind has released comprehensive pre-trained Sparse Autoencoders for both variants through the Gemma Scope project [12], eliminating the need for SAE training and allowing the study to focus on the novel analysis contribution. Third, the 2B parameter scale is computationally tractable for systematic analysis across all layers while remaining large enough to exhibit the complex feature representations observed in prior interpretability work [22].

2.2.2 Sparse Autoencoders: Gemma Scope

The Sparse Autoencoders used in this study are drawn from the Gemma Scope suite [12], a comprehensive collection of open-source SAEs trained on Gemma 2 by Google DeepMind. Gemma Scope provides pre-trained JumpReLU SAEs [21] for every layer and sublayer of both the base and instruction-tuned Gemma 2 2B and 9B models.

For the primary analysis, SAEs with a dictionary width of 16,384 features (approximately $7\times$ the model’s hidden dimension of 2304) are used across all 26 layers, targeting the residual stream activation site. This width was selected to balance feature granularity against analytical tractability: larger dictionaries (32K, 65K, or 131K features, also available in Gemma Scope) capture increasingly fine-grained feature splits but produce proportionally more features to match, label, and interpret. Where the 16K analysis reveals evidence of feature splitting—multiple similar features that appear to decompose a single concept—selected analyses are repeated at 32K to verify whether finer resolution changes the conclusions.

All Gemma Scope SAEs for the 2B model were trained using the JumpReLU activation function on approximately 4 billion tokens of web text sampled from a distribution similar to

Gemma 2’s pre-training data. Each SAE was trained independently on activations from its respective model variant (base or IT), meaning the base-model SAEs and the IT-model SAEs have separate learned dictionaries. This independence is both a strength and a limitation: it avoids any architectural coupling between the two feature sets, but it also means that features representing the same underlying concept in different models may be learned as distinct dictionary elements, necessitating explicit feature matching (see Section 2.6.2).

Key quality metrics reported by Lieberum et al. [12] for the 16K SAEs include a mean L_0 (average number of active features per token) in the range of 30–60 depending on the layer and sparsity configuration, and fraction of variance unexplained (FVU) values that indicate the SAEs capture the majority of activation variance while maintaining sparsity. The sparsity-fidelity trade-off for the selected SAE configurations is reported in the original Gemma Scope paper and is not re-evaluated here, as the focus of this study is on comparative feature analysis rather than SAE quality assessment.

2.2.3 Text Corpora

Two text datasets serve complementary roles in the analysis pipeline.

FineWeb [20] is a large-scale, deduplicated English web text corpus released by HuggingFace. It serves as the primary dataset for broad activation collection and feature-level comparison. FineWeb was selected because its distribution closely matches the pre-training data of both the Gemma 2 models and the Gemma Scope SAEs, ensuring that the SAEs operate within their intended distribution and that reconstruction quality remains high. From the full corpus, a random sample of 50,000 sequences is drawn, each truncated to 256 tokens, yielding approximately 12.8 million tokens of activation data. The same token sequences are processed by both the base and IT models to ensure that feature activations are directly comparable at each token position.

Instruction-Following Evaluation Set. To specifically investigate features relevant to the behavioural differences between base and instruction-tuned models, a secondary evaluation set is constructed by sampling from existing instruction-following benchmarks. This set draws from Alpaca-Eval [11] for general instruction-following prompts, TruthfulQA [13] for prompts

probing factual accuracy and truthfulness, and a curated subset of safety-relevant prompts from publicly available alignment evaluation datasets. The purpose of this set is to probe feature behaviour in contexts where the two model variants are expected to diverge most—when presented with instructions, sensitive topics, or requests that the IT model has been specifically trained to handle differently from the base model. Approximately 2,000 prompts are included, categorised by type (general instruction, factual, safety-relevant, creative).

A known limitation of this two-dataset design is that the FineWeb corpus does not include conversational or instruction-formatted text, which is the domain where post-training effects are most pronounced. The instruction-following set compensates for this gap, but its smaller size limits the statistical power of comparisons drawn from it. This trade-off is discussed further in Section 2.8.

2.3 Data Processing and Activation Collection Pipeline

The activation collection pipeline extracts internal representations from both models in a controlled, reproducible manner, as illustrated in Figure 1. For each input sequence, the pipeline records the residual stream activations at each of the 26 layers, producing paired activation vectors for the base and IT models on identical inputs.

2.3.1 Tokenization and Input Preparation

All input sequences are tokenized using Gemma 2’s shared tokenizer (vocabulary size: 256,000). Because both models share the same tokenizer, the same input text produces identical token sequences for both models, ensuring token-level alignment of activations. Sequences from FineWeb are truncated to 256 tokens; sequences shorter than 256 tokens are discarded rather than padded, to avoid introducing padding-related artifacts into the activation distributions.

For the instruction-following evaluation set, prompts are formatted as raw text without applying any chat template or special tokens. This choice ensures that both models receive identical inputs—applying the IT model’s chat template would introduce tokens that the base model has never seen, confounding the comparison. The consequence of this choice is that the IT model may not produce its best instruction-following behaviour, since it was fine-tuned to

expect a specific prompt format. However, this study is concerned with internal feature representations rather than output quality, and the raw-format approach guarantees that any observed activation differences arise from learned representational changes rather than input formatting differences.

2.3.2 Activation Extraction

For each input sequence and each of the 26 transformer layers, the residual stream activation vector $\mathbf{h}_l \in \mathbb{R}^{2304}$ is recorded after the layer’s output has been added to the residual stream. This site captures the cumulative representation at each layer, incorporating the contributions of all preceding attention and MLP computations.

Activations are extracted using TransformerLens [17], which provides hooks into the model’s forward pass at each layer. Both models are run in inference mode with no gradient computation. All activations are stored in 32-bit floating point precision to match the precision used during Gemma Scope SAE training.

2.3.3 SAE Feature Decomposition

Each activation vector $\mathbf{h}_l$ is passed through the corresponding Gemma Scope SAE for that layer and model variant. The SAE decomposes the activation into a sparse feature vector $\mathbf{f}_l \in \mathbb{R}^{16384}$, where the vast majority of entries are zero and the non-zero entries represent the activation strength of each detected feature.

Formally, the JumpReLU SAE computes:

$$\mathbf{f}_l = \text{JumpReLU}_\theta(W_{\text{enc}}(\mathbf{h}_l - \mathbf{b}_{\text{pre}}) + \mathbf{b}_{\text{enc}}) \quad (1)$$

where $W_{\text{enc}} \in \mathbb{R}^{16384 \times 2304}$ is the encoder weight matrix, $\mathbf{b}_{\text{pre}} \in \mathbb{R}^{2304}$ is a pre-encoder bias (typically set to the mean activation), $\mathbf{b}_{\text{enc}} \in \mathbb{R}^{16384}$ is the encoder bias, and JumpReLU_θ is a discontinuous activation function defined as:

$$\text{JumpReLU}_\theta(z_i) = z_i \cdot \mathbb{I}[z_i > \theta_i] \quad (2)$$

Here, θ_i is a learned threshold for each feature i . Unlike the standard ReLU, which activates for any positive input, JumpReLU only activates when the pre-activation exceeds a learned threshold, producing exactly zero output otherwise. This yields sparser representations without the shrinkage artifacts associated with L_1 -penalised ReLU SAEs [21].

The reconstructed activation is then:

$$\hat{\mathbf{h}}_l = W_{\text{dec}} \mathbf{f}_l + \mathbf{b}_{\text{pre}} \quad (3)$$

where $W_{\text{dec}} \in \mathbb{R}^{2304 \times 16384}$ is the decoder weight matrix. Each column $\mathbf{d}_i$ of W_{dec} represents the direction in activation space associated with feature i , and is normalised to unit norm during training. These decoder directions are central to the feature matching procedure described in Section 2.6.2.

The output of this stage is, for each input token, a pair of sparse feature vectors—one from the base model’s SAE and one from the IT model’s SAE—along with the corresponding decoder direction matrices. These constitute the raw data for all subsequent analyses.

2.4 Exploratory Analysis of Raw Activations

Before applying the SAE-based feature comparison framework, a preliminary analysis characterises how the raw activation spaces of the two models relate to each other. This exploratory stage serves two purposes: it establishes baseline expectations for the magnitude and distribution of representational differences, and it motivates the need for the more fine-grained, feature-level analysis that follows.

2.4.1 Activation Norm Comparison

For each layer l , the mean activation norm $\mathbb{E}[\|\mathbf{h}_l\|]$ is computed across all tokens in the FineWeb sample, separately for each model. Systematic differences in activation norms across layers may indicate that post-training amplifies or dampens representations at certain depths. Prior work has observed that instruction-tuned models sometimes exhibit higher activation norms in later layers, consistent with stronger behavioural signals being injected during post-training

[5].

2.4.2 Representational Similarity

To quantify how similar the overall activation geometries are between the two models, Centered Kernel Alignment (CKA) is computed layer-by-layer. CKA [10] measures the similarity between two sets of representations by comparing their pairwise similarity structures, and is invariant to orthogonal transformations and isotropic scaling. Given activation matrices $X \in \mathbb{R}^{n \times d}$ and $Y \in \mathbb{R}^{n \times d}$ (rows are tokens, columns are dimensions) from the base and IT models respectively, linear CKA is computed as:

$$\text{CKA}(X, Y) = \frac{\|Y^\top X\|_F^2}{\|X^\top X\|_F \cdot \|Y^\top Y\|_F} \quad (4)$$

where $\|\cdot\|_F$ denotes the Frobenius norm. CKA values range from 0 (completely dissimilar representations) to 1 (identical up to linear transformation). A layer-by-layer CKA profile reveals where in the model the two variants diverge most, guiding the selection of layers for detailed microscopic analysis.

This exploratory analysis is expected to show high CKA values in early layers (where representations are primarily determined by pre-training and capture low-level lexical and positional information) and progressively lower values in later layers (where post-training has the most opportunity to reshape representations for behavioural alignment). However, CKA operates on the aggregate activation space and cannot identify which specific features changed or how—motivating the SAE-based decomposition that follows.

2.5 Sparse Autoencoder Framework

This section formally defines the Sparse Autoencoder framework and the theoretical assumptions that justify its use as an interpretability tool for comparing feature representations across model variants.

2.5.1 Theoretical Motivation

The application of Sparse Autoencoders to this problem rests on two foundational hypotheses from the mechanistic interpretability literature.

The **linear representation hypothesis** posits that neural networks represent meaningful concepts as directions in their activation space [15, 19]. Under this view, concepts such as “formal language,” “Python code,” or “safety-relevant content” correspond to specific linear directions along which activations vary. If this hypothesis holds, then the feature representations extracted by SAEs—which are precisely linear directions in activation space—capture genuine conceptual structure rather than artifacts of the decomposition method.

The **superposition hypothesis** [6] proposes that neural networks represent far more features than they have dimensions, by encoding features as nearly-orthogonal directions that overlap in the activation space. This explains why individual neurons are often polysemantic (responding to multiple unrelated concepts): each neuron participates in the representation of many superimposed features. SAEs address this by projecting activations into a much higher-dimensional space (16,384 features from 2,304 dimensions) where each basis direction ideally corresponds to a single interpretable concept—a property known as monosemanticity [4].

Together, these hypotheses provide the theoretical basis for comparing features across models. If both the base and IT models represent concepts as linear directions, and if SAEs can recover these directions, then comparing SAE features across models amounts to comparing the conceptual vocabularies that each model has learned to use. Features present in both models represent shared conceptual structure inherited from pre-training; features present only in the IT model may represent concepts introduced or made salient by post-training; and features present only in the base model may represent concepts that post-training suppresses or reorganises.

2.5.2 Assumptions and Caveats

Several assumptions underlie this framework, and their potential violations should be acknowledged.

First, the linear representation hypothesis, while supported by substantial empirical evidence [15, 19], is not guaranteed to hold for all features. Some concepts may be represented

non-linearly, in which case SAEs would fail to capture them. This study cannot detect features that violate linearity.

Second, SAE reconstruction is imperfect. The fraction of activation variance left unexplained by the SAE (FVU) means that some information in the original activations is lost. Features that contribute to the unexplained variance are invisible to this analysis. The use of Gemma Scope SAEs with documented quality metrics [12] mitigates but does not eliminate this concern.

Third, the independence of the two SAE dictionaries means that the absence of a matching feature across models does not necessarily mean the underlying concept is absent—it may simply be decomposed differently. The multi-criteria matching approach described in Section 2.6.2 is designed to reduce false negatives, but some genuine correspondences will inevitably be missed.

Fourth, the analysis is limited to the residual stream activation site. Features represented primarily in attention patterns or MLP intermediate activations may not be captured. This scope limitation is a deliberate trade-off for tractability, and extending the analysis to other sites is noted as future work.

2.6 Feature Comparison Methodology

This section describes the core analytical contribution of the thesis: a systematic pipeline for identifying which features are preserved, created, or modified between the base and instruction-tuned models. The pipeline, illustrated in Figure 2, proceeds from global similarity metrics through feature-level matching to thematic categorisation.

2.6.1 Macroscopic Analysis: Feature Landscape Overview

The first level of analysis characterises the overall similarity of the feature dictionaries across models. For each layer l , the Mean Maximum Cosine Similarity (MMCS) between the base and IT model SAE dictionaries is computed. Let $D_l^{\text{base}} \in \mathbb{R}^{2304 \times 16384}$ and $D_l^{\text{IT}} \in \mathbb{R}^{2304 \times 16384}$ denote the decoder weight matrices of the base and IT SAEs at layer l , with columns $\mathbf{d}_i^{\text{base}}$ and

$\mathbf{d}_j^{\text{IT}}$ representing the decoder directions for each feature. The MMCS from base to IT is:

$$\text{MMCS}(D_l^{\text{base}}, D_l^{\text{IT}}) = \frac{1}{N} \sum_{i=1}^N \max_j \cos(\mathbf{d}_i^{\text{base}}, \mathbf{d}_j^{\text{IT}}) \quad (5)$$

where $N = 16,384$ is the number of features and $\cos(\cdot, \cdot)$ denotes cosine similarity. A high MMCS indicates that most features in the base dictionary have a close geometric counterpart in the IT dictionary. The MMCS is computed in both directions (base $\rightarrow$ IT and IT $\rightarrow$ base), as the mapping need not be symmetric.

Additionally, the distribution of maximum cosine similarities is examined—not just its mean. A bimodal distribution (with a cluster near 1.0 and a cluster near 0.5) would suggest that some features are well-preserved while others are substantially altered, whereas a unimodal distribution near 0.8 would suggest gradual, uniform drift.

The MMCS profile across layers is expected to mirror the CKA profile from Section 2.4.2: higher similarity in early layers, decreasing toward later layers where post-training effects accumulate.

2.6.2 Feature Matching

Individual features are matched across models using two complementary criteria.

Decoder direction similarity. For each feature i in the base model SAE, the cosine similarity between its decoder direction $\mathbf{d}_i^{\text{base}}$ and every decoder direction $\mathbf{d}_j^{\text{IT}}$ in the IT model SAE is computed. The highest-similarity IT feature is identified as the candidate match. A match is accepted if the cosine similarity exceeds a threshold τ_d . The threshold $\tau_d = 0.8$ is adopted as the primary criterion, following conventions in the SAE comparison literature, with sensitivity analyses conducted at $\tau_d \in \{0.7, 0.85, 0.9\}$ to assess robustness.

Activation pattern correlation. As a complementary criterion, the Pearson correlation between the activation patterns of candidate feature pairs is computed over the shared FineWeb evaluation set. Let $\mathbf{a}_i^{\text{base}} \in \mathbb{R}^T$ and $\mathbf{a}_j^{\text{IT}} \in \mathbb{R}^T$ be the vectors of activation values for features i and j across T tokens. The activation correlation is:

$$r_{ij} = \text{corr}(\mathbf{a}_i^{\text{base}}, \mathbf{a}_j^{\text{IT}}) \quad (6)$$

A match is considered high-confidence if both $\cos(\mathbf{d}_i^{\text{base}}, \mathbf{d}_j^{\text{IT}}) > \tau_d$ and $r_{ij} > \tau_a$, where $\tau_a = 0.7$ is adopted as the activation correlation threshold. Features matching on decoder similarity alone (geometric match but different activation patterns) or activation correlation alone (similar firing patterns but different geometric directions) are flagged for closer inspection, as these may indicate features that have been functionally preserved but geometrically reorganised, or vice versa.

2.6.3 Feature Classification

Based on the matching results, features are classified into four categories:

Preserved features have a high-confidence match (both criteria exceeded) between the base and IT models. These represent conceptual structure inherited from pre-training that post-training leaves intact.

Modified features have a partial match: they exceed one matching criterion but not both, or their matched counterpart shows a significantly different activation magnitude distribution. These may represent concepts that post-training has adjusted—for instance, a feature detecting “requests for harmful content” that exists in both models but activates more strongly in the IT model, suggesting post-training amplified its salience.

Base-exclusive features have no match exceeding either threshold in the IT model’s dictionary. These represent either concepts that post-training has suppressed or features that have been reorganised beyond recognition.

IT-exclusive features have no match in the base model’s dictionary. These are candidates for newly created features—concepts that the post-training process introduced.

The proportion of features in each category, computed per layer and aggregated across the model, constitutes the primary quantitative result of the macroscopic analysis.

2.6.4 Automated Interpretability Pipeline

To understand what the matched and unmatched features represent, an automated interpretability pipeline generates and validates natural language descriptions for selected features.

Feature selection. Not all 16,384 features per layer are labelled. Features are selected

for labelling based on their relevance to the research questions: all IT-exclusive features with non-trivial activation frequency (firing on at least 0.1% of tokens), all base-exclusive features with non-trivial activation frequency, a random sample of preserved features for baseline characterisation, and modified features with the largest activation magnitude differences between models.

Description generation. For each selected feature, the top 20 text sequences from the FineWeb sample that produce the highest activation values are collected, along with 10 randomly sampled sequences where the feature does not activate. These examples are presented to a language model (Claude, Anthropic) with the following prompt structure: the activated tokens are highlighted within each sequence, the non-activating examples are provided as contrast, and the model is asked to produce a concise description of the pattern that distinguishes activating from non-activating contexts.

Description validation. Generated descriptions are validated using two scoring methods adapted from the automated interpretability literature [2, 22].

Detection scoring presents a separate LLM instance with the feature description and a mixed set of activating and non-activating sequences, then asks it to classify which sequences would activate the feature. The detection score is the proportion of correctly classified sequences, where scores above 0.7 indicate that the description captures the feature’s behaviour with reasonable accuracy.

Fuzzing scoring presents the description and an activating sequence, then asks the LLM to predict which specific tokens within the sequence trigger the feature. This token-level validation is a stricter test that identifies descriptions that are directionally correct but insufficiently precise.

Features whose descriptions fail validation (detection score below 0.5) are flagged as poorly understood and excluded from thematic categorisation, though they are retained in quantitative counts.

Thematic categorisation. Validated feature descriptions are manually grouped into thematic categories: language and syntax (grammatical structures, punctuation patterns, formatting), knowledge and semantics (factual concepts, domain-specific content, named entities),

instruction and style (response formatting, conversational conventions, hedging language), and safety and alignment (refusal patterns, sensitivity to harmful content, bias-related activations). This categorisation enables the analysis to answer not just how many features change, but what kinds of features are most affected by post-training.

2.6.5 Methodological Extension: Crosscoders for Joint Feature Extraction

The feature matching approach described above relies on independently trained SAEs whose dictionaries are compared post-hoc. While practical and well-grounded, this approach has a fundamental limitation: two SAEs trained independently on different activation distributions may decompose the same underlying concept into different dictionary elements, or may split a single concept differently across multiple features. This means that some genuine feature correspondences will be missed by the matching procedure, and the counts of “exclusive” features may overestimate the true number of features unique to each model.

An alternative methodology that addresses this limitation is the crosscoder framework [14]. A crosscoder is a variant of the Sparse Autoencoder that jointly encodes activations from both the base and instruction-tuned models using a single shared dictionary. Concretely, a crosscoder receives the concatenated activation vectors $[\mathbf{h}_l^{\text{base}}; \mathbf{h}_l^{\text{IT}}] \in \mathbb{R}^{2 \times 2304}$ from the same input processed by both models, and produces a single sparse feature vector $\mathbf{f}_l \in \mathbb{R}^K$. Each feature then decodes separately to each model’s activation space through distinct decoder columns $\mathbf{d}_i^{\text{base}}$ and $\mathbf{d}_i^{\text{IT}}$. A feature is naturally classified as shared if both decoder columns have substantial norm, and as model-exclusive if one decoder column dominates.

This design eliminates the need for post-hoc feature matching entirely, because both models are explained through the same dictionary from the outset. Recent work has further refined this approach through Dedicated Feature Crosscoders (DFCs), which explicitly partition the dictionary into shared and model-specific feature slots, encouraging cleaner separation and reducing the risk of misattribution [1]. Additionally, research on crosscoder training dynamics has shown that the choice of sparsity penalty matters: standard L_1 regularisation can cause shared features to be incorrectly attributed as model-exclusive, whereas BatchTopK loss substantially mitigates this issue by enforcing sparsity at the batch level rather than per-example [16].

This study begins with the separate-SAE approach for three reasons. First, the Gemma Scope SAEs are pre-trained, validated, and publicly available, allowing the analysis to proceed without the computational cost and hyperparameter sensitivity of training crosscoders from scratch. Second, using established, independently validated SAEs provides a conservative and transparent baseline—the matching criteria and thresholds are explicit and can be scrutinised. Third, the separate-SAE approach is more directly comparable to prior work on SAE-based model analysis [12, 22], facilitating interpretation of results in the context of existing literature.

Should the separate-SAE analysis reveal a substantial proportion of ambiguously matched features—features that meet one matching criterion but not the other, suggesting they may have been reorganised rather than truly created or lost—a crosscoder-based analysis on selected layers would be pursued as a complementary method. Training crosscoders on 3–4 representative layers (early, middle, and late) using the Language-Model-SAEs framework [7] with BatchTopK loss would allow direct comparison of results between the two methodological approaches. Convergent findings across both approaches would strengthen confidence in the conclusions; divergent findings would highlight the sensitivity of results to methodological choices, which itself would be a valuable contribution.

2.7 Causal Verification via Feature Steering

The feature comparison methodology identifies correlational differences between models—features that activate differently. As shown in Figure 3, to establish that these differences are causally meaningful, selected features are subjected to activation steering experiments that test whether manipulating a feature’s activation actually changes model behaviour in the predicted direction. The results of these experiments, combined with the quantitative metrics from the feature classification stage, converge into an overall assessment of the wrapper hypothesis.

2.7.1 Activation Steering Method

For a given feature i with decoder direction $\mathbf{d}_i$ and a target activation magnitude α , the steering intervention modifies the model’s residual stream activation at layer l during a forward pass:

$$\mathbf{h}'_l = \mathbf{h}_l + \alpha \cdot \mathbf{d}_i \quad (7)$$

This adds the feature’s direction to the activation, simulating an increase in the feature’s activation strength (for positive α) or a suppression (for negative α). The modified activation propagates through the remaining layers, and the resulting change in model output (measured by generation quality, token probabilities, or behavioural classification) indicates the feature’s causal role.

2.7.2 Case Study Selection

Approximately 5–10 features are selected for causal verification based on their relevance to the research questions:

Features that are IT-exclusive and whose descriptions suggest instruction-following or safety-related behaviour are tested by steering them in the base model. If activating an IT-exclusive feature in the base model causes it to produce more instruction-following or safety-aware outputs, this provides evidence that the feature genuinely encodes the relevant behaviour and was introduced by post-training.

Features that are base-exclusive and whose descriptions suggest capabilities or knowledge that may have been suppressed are tested by steering them in the IT model. If re-activating a suppressed feature changes the IT model’s behaviour, this suggests that the underlying capability persists but is actively inhibited by post-training—consistent with the “wrapper hypothesis” [9].

Modified features whose activation patterns shifted between models are tested at various steering magnitudes to characterise the relationship between feature strength and behavioural change.

2.7.3 Evaluation of Steering Effects

Steering effects are evaluated both quantitatively and qualitatively. Quantitatively, the KL divergence between the steered and unsteered output distributions is computed to measure the magnitude of the behavioural change. Qualitatively, generated text samples under steering are examined to assess whether the behavioural change aligns with the feature’s description—for instance, whether steering a “formal language” feature actually makes outputs more formal.

2.8 Assumptions, Limitations, and Scope

The methodology rests on several assumptions and is subject to limitations that constrain the interpretation of results.

Assumptions. The linear representation hypothesis is assumed to hold sufficiently for the features of interest. The Gemma Scope SAEs are assumed to provide adequate reconstruction quality for the analysis to be meaningful. The matching thresholds ($\tau_d = 0.8$, $\tau_a = 0.7$) are assumed to appropriately distinguish genuine feature correspondences from coincidental similarity. Each of these assumptions is tested indirectly: the linear representation hypothesis through the coherence of automated feature labels, SAE quality through reported FVU values, and matching thresholds through sensitivity analysis at alternative values.

Limitations. The study examines a single model family (Gemma 2) at a single scale (2B parameters). Results may not generalise to other architectures (e.g., mixture-of-experts models) or larger scales, where both feature representations and the effects of post-training may differ qualitatively. The analysis is restricted to residual stream activations and does not examine attention patterns or MLP internals. The use of independently trained SAEs introduces unavoidable noise in feature matching—some genuine correspondences will be missed and some spurious matches will be accepted. The instruction-following evaluation set, while curated to cover diverse prompt types, cannot exhaustively represent the full distribution of inputs where post-training effects manifest. Finally, the study compares only the final pre-trained and post-trained checkpoints and cannot distinguish the individual contributions of SFT and RLHF within the post-training pipeline, as intermediate checkpoints are not publicly available.

Scope. The results are valid for the specific model pair examined (Gemma 2 2B base and IT) and the specific SAE configuration used (Gemma Scope 16K JumpReLU on residual stream). Claims about “post-training effects on feature representations” should be understood as conditional on these choices. The discussion chapter addresses the extent to which findings may generalise based on convergent evidence from prior work on related models and methods.

Sensitivity. The primary results that could be invalidated include: the feature preservation rate (sensitive to matching thresholds—tested at multiple values), the identification of IT-exclusive features (sensitive to SAE dictionary completeness—could reflect features the base SAE failed to learn rather than genuinely absent features), and the causal verification results (sensitive to steering magnitude and the specific prompts used—tested across a range of magnitudes and prompt types).

2.9 Reproducibility

All experiments are conducted in Python 3.11 using the following software stack:

- **TransformerLens** [17] for model loading and activation extraction
- **SAELens** [3] for loading Gemma Scope SAEs, computing feature activations, and performing activation steering
- **PyTorch 2.x** as the underlying deep learning framework
- **NumPy** and **SciPy** for numerical computation and statistical tests
- **Anthropic API** (Claude) for automated feature labelling and validation

All experiments are executed on Google Colab Pro, using NVIDIA A100 40GB GPU instances with up to 52GB of system RAM. As GPU allocation in Colab Pro may vary across sessions, the specific hardware configuration is logged at the start of each experiment run. Random seeds are fixed at 42 for all stochastic operations, including data sampling and the ordering of evaluation prompts. The FineWeb and instruction-following evaluation samples, along with all analysis code, feature activation caches, feature labels, and matching results, are made available in a public GitHub repository.

The full pipeline—from activation extraction through feature matching, labelling, and causal verification—is implemented as a series of reproducible scripts with configuration files specifying all hyperparameters. An informed reader should be able to reproduce the complete analysis by cloning the repository, downloading the publicly available models and SAEs, and executing the pipeline scripts in sequence.

A Literature Review Summary

This appendix summarises the studies that most directly motivate the research design and clarifies how each contributes to the present project.

- **Elhage et al. (2022)** [6] establish the superposition framework, explaining why individual neurons are often polysemantic and why neuron-level interpretability is limited.
- **Templeton et al. (2024)** [22] demonstrate that sparse autoencoders can recover large numbers of interpretable, causally meaningful features from frontier language models, making feature-level comparison feasible.
- **Ouyang et al. (2022)** [18] show that post-training techniques such as supervised fine-tuning and RLHF can strongly improve instruction-following, helpfulness, and safety.
- **Zhou et al. (2023)** [23] argue that most capability is learned during pre-training and that a relatively small amount of fine-tuning data can substantially reshape model behaviour, motivating the superficial alignment hypothesis.
- **Du et al. (2025)** [5] provide direct mechanistic evidence that post-training affects some behavioural directions, especially refusal, more than core factual knowledge representations.

B Implications for the Present Study

The literature review motivates the methodology in three ways.

- It justifies the use of sparse autoencoders as the main interpretability tool, because superposition makes individual neurons an unreliable unit of analysis.
- It motivates comparing a base model with its instruction-tuned counterpart, because post-training is the stage where alignment-relevant behavioural changes are introduced.

- It frames the core empirical question of the project: whether post-training creates new interpretable features, suppresses existing ones, or primarily changes the activation patterns of a largely preserved feature set.

References

- [1] Anonymous. *Dedicated Feature Crosscoders*. OpenReview preprint. 2025. URL: <https://openreview.net/forum?id=dedicated-feature-crosscoders>.
- [2] Steven Bills et al. *Language models can explain neurons in language models*. 2023. URL: <https://openaipublic.blob.core.windows.net/neuron-explainer/paper/index.html>.
- [3] Joseph Bloom et al. *SAELens: SAE Training and Analysis Library*. 2024. URL: <https://github.com/jbloomAus/SAELens>.
- [4] Trenton Bricken et al. *Towards Monosemanticity: Decomposing Language Models With Dictionary Learning*. 2023. URL: <https://transformer-circuits.pub/2023/monosemantic-features/>.
- [5] Hongzhe Du et al. *How Post-Training Reshapes LLMs: A Mechanistic View on Knowledge, Truthfulness, Refusal, and Confidence*. version: 2. Aug. 11, 2025. DOI: 10.48550/arXiv.2504.02904. arXiv: 2504.02904[cs]. URL: <http://arxiv.org/abs/2504.02904> (visited on 2026).
- [6] Nelson Elhage et al. *Toy Models of Superposition*. Sept. 21, 2022. DOI: 10.48550/arXiv.2209.10652. arXiv: 2209.10652[cs]. URL: <http://arxiv.org/abs/2209.10652> (visited on 2026).
- [7] Xuyang Ge et al. *Language-Model-SAEs: A Framework for Training and Evaluating Sparse Autoencoders*. 2024. URL: <https://github.com/OpenMOSS/Language-Model-SAEs>.
- [8] Gemma Team et al. *Gemma 2: Improving Open Language Models at a Practical Size*. 2024. DOI: 10.48550/arXiv.2408.00118. arXiv: 2408.00118[cs]. URL: <http://arxiv.org/abs/2408.00118>.
- [9] Samyak Jain et al. *Mechanistically analyzing the effects of fine-tuning on procedurally defined tasks*. Aug. 21, 2024. DOI: 10.48550/arXiv.2311.12786. arXiv: 2311.12786[cs]. URL: <http://arxiv.org/abs/2311.12786> (visited on 2026).

- [10] Simon Kornblith et al. *Similarity of Neural Network Representations Revisited*. July 19, 2019. DOI: 10.48550/arXiv.1905.00414. arXiv: 1905.00414[cs]. URL: <http://arxiv.org/abs/1905.00414> (visited on 2026).
- [11] Xuechen Li et al. *AlpacaEval: An Automatic Evaluator of Instruction-Following Models*. 2023. URL: https://github.com/tatsu-lab/alpaca_eval.
- [12] Tom Lieberum et al. *Gemma Scope: Open Sparse Autoencoders Everywhere All At Once on Gemma 2*. Aug. 19, 2024. DOI: 10.48550/arXiv.2408.05147. arXiv: 2408.05147[cs]. URL: <http://arxiv.org/abs/2408.05147> (visited on 2026).
- [13] Stephanie Lin, Jacob Hilton, and Owain Evans. *TruthfulQA: Measuring How Models Mimic Human Falsehoods*. 2022. DOI: 10.48550/arXiv.2109.07958. arXiv: 2109.07958[cs]. URL: <http://arxiv.org/abs/2109.07958>.
- [14] Jack Lindsey et al. *Crosscoders*. 2024. URL: <https://transformer-circuits.pub/2024/crosscoders/index.html>.
- [15] Tomas Mikolov et al. *Efficient Estimation of Word Representations in Vector Space*. Sept. 7, 2013. DOI: 10.48550/arXiv.1301.3781. arXiv: 1301.3781[cs]. URL: <http://arxiv.org/abs/1301.3781> (visited on 2026).
- [16] Jonas Minder et al. *On Feature Attributions in Crosscoders*. 2025. DOI: 10.48550/arXiv.2502.01819. arXiv: 2502.01819[cs]. URL: <http://arxiv.org/abs/2502.01819>.
- [17] Neel Nanda and Joseph Bloom. *TransformerLens*. 2022. URL: <https://github.com/TransformerLensOrg/TransformerLens>.
- [18] Long Ouyang et al. *Training language models to follow instructions with human feedback*. Mar. 4, 2022. DOI: 10.48550/arXiv.2203.02155. arXiv: 2203.02155[cs]. URL: <http://arxiv.org/abs/2203.02155> (visited on 2026).
- [19] Kiho Park, Yo Joong Choe, and Victor Veitch. *The Linear Representation Hypothesis and the Geometry of Large Language Models*. July 17, 2024. DOI: 10.48550/arXiv.2311.03658. arXiv: 2311.03658[cs]. URL: <http://arxiv.org/abs/2311.03658> (visited on 2026).

- [20] Guilherme Penedo et al. *The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale*. 2024. DOI: 10.48550/arXiv.2406.17557. arXiv: 2406.17557[cs]. URL: <http://arxiv.org/abs/2406.17557>.
- [21] Senthooran Rajamanoharan et al. *Improving Dictionary Learning with Gated Sparse Autoencoders*. Apr. 30, 2024. DOI: 10.48550/arXiv.2404.16014. arXiv: 2404.16014[cs]. URL: <http://arxiv.org/abs/2404.16014> (visited on 2026).
- [22] Adly Templeton et al. *Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet*. 2024. URL: <https://transformer-circuits.pub/2024/scaling-monosemanticity/>.
- [23] Chunting Zhou et al. *LIMA: Less Is More for Alignment*. May 18, 2023. DOI: 10.48550/arXiv.2305.11206. arXiv: 2305.11206[cs]. URL: <http://arxiv.org/abs/2305.11206> (visited on 2026).